
ANEXO DEL ENTREGABLE N.º 1

Guía de uso del *pipeline* de datos `smn_toe`

Concepto:

Servicio de asistencia en temas de investigación para evaluar la influencia del cambio climático en los eventos de El Niño y sus impactos en el Perú

Prestado para: Servicio Nacional de Meteorología e Hidrología del Perú – SENAMHI

Este anexo acompaña al informe del Entregable 1. Explica cómo instalar y ejecutar el *pipeline*, qué produce, cuánto tarda y cómo se usarán sus resultados en los entregables siguientes. Está escrito para quien usa Linux de forma básica: cada comando se puede copiar tal cual. El repositorio es de libre acceso.

Responsable:

Ing. Jonathan Aaron Paredes Quispe

FECHA: 28 de septiembre de 2026

Índice

1. Qué hace el <i>pipeline</i>	2
2. Qué se necesita	2
3. Instalación (una sola vez)	2
4. Ejecutar	3
4.1. La forma general	3
4.2. Elegir el comando	4
4.3. Dejarlo corriendo en segundo plano	4
4.4. Qué aparece en pantalla	4
5. Cuánto tarda	5
6. Dónde quedan los resultados	6
6.1. Cómo revisar los resultados	6
7. Qué hace cada paso (detalle técnico)	6
7.1. Paso 00 – <code>scripts/00_setup_env.sh</code>	7
7.2. Paso 00b – <code>scripts/00b_build_model_list.py</code>	7
7.3. Paso 00c – <code>scripts/00c_check_paper_models.py</code> (diagnóstico manual)	7
7.4. Paso 01 – <code>scripts/01_query_esgf_catalog.py</code>	7
7.5. Paso 02 – descarga en cascada	7
7.6. Paso 03 – <code>scripts/03_download_ersstv5.sh</code>	8
7.7. Paso 04 – <code>scripts/04_process_to_common_grid.sh</code>	8
7.8. Paso 05 – <code>scripts/05_apply_ocean_mask.py</code>	9
7.9. Paso 06 – <code>scripts/06_qc_checks.py</code>	9
7.10. Paso 07 – <code>scripts/07_build_inventory_report.py</code>	9
7.11. Reporte de estado – <code>scripts/generate_status_report.py</code>	9
8. Problemas frecuentes	10
9. Qué se espera hacer con estos datos	10
10. Estructura del repositorio	11

1. Qué hace el *pipeline*

El *pipeline* reúne en un único formato comparable la temperatura superficial del mar (TSM) de los modelos climáticos CMIP6 y la observación ERSSTv5. Descarga los datos de internet, los lleva a una misma grilla, calendario, unidad (°C) y máscara de océano, y verifica que estén completos. Todo se hace con un solo comando, `./run.sh`. La Figura 1 resume el recorrido.

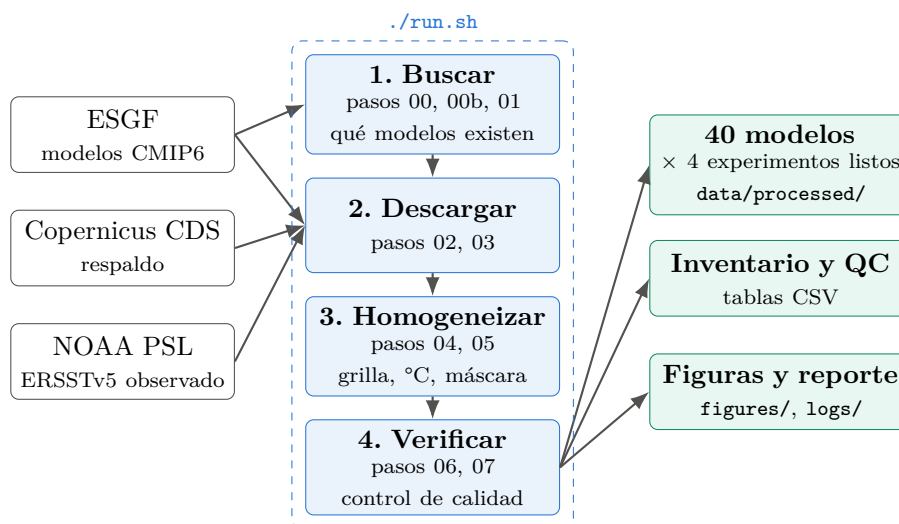


Figura 1: Recorrido de los datos: tres fuentes en internet, cuatro etapas automáticas dentro de `run.sh` y tres tipos de producto.

Los productos alimentan directamente el Entregable 2. Los números de paso (00 a 07) son los que usa el informe y los que acepta `run.sh` para ejecutar solo una parte (Sección 4).

2. Qué se necesita

Requisito	Detalle
Sistema operativo	Linux (probado en Ubuntu y en un clúster HPC).
Espacio en disco	Unos 90 GB libres : 73 GB de datos crudos, cerca de 9 GB de datos procesados e intermedios y margen. Se comprueba con <code>df -h</code> .
Conexión a internet	Estable. La descarga completa toma del orden de 13 horas.
Conda (Miniconda)	Gestor de programas; instala Python, CDO y el resto de las dependencias (Sección 3).
Repositorio	https://github.com/Japq91/smn_toe , de libre acceso.
Cuenta de Copernicus	Opcional . Solo se usa para los modelos que ESGF no resuelve; sin ella, ese paso se omite y el resto sigue.

3. Instalación (una sola vez)

Abra una terminal (en Ubuntu: **Ctrl+Alt+T**) y siga estos pasos en orden.

Paso 1. Instalar Miniconda (omitir si el comando `conda` ya existe; se comprueba con `conda --version`):

```
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
bash Miniconda3-latest-Linux-x86_64.sh
```

Acepte la licencia y las opciones por defecto. Al terminar, **cierre la terminal y abra una nueva**.

Paso 2. Descargar el repositorio y entrar a su carpeta:

```
git clone https://github.com/Japq91/smn_toe.git
cd smn_toe
```

Paso 3. Crear el entorno de programas (tarda unos minutos y se hace una sola vez):

```
conda env create -f environment.yml
```

Esto crea el entorno `smn_toe` con Python 3.11, CDO 2.1.1, `netCDF4`, `xarray`, `matplotlib`, `cartopy` y `cdsapirc`. No hace falta activarlo a mano: `run.sh` lo activa solo.

Paso 4 (opcional). Credenciales de Copernicus. Cree una cuenta en <https://cds.climate.copernicus.eu>, acepte la licencia del conjunto `projections-cmip6` y guarde su clave personal en el archivo `~/.cdsapirc`:

```
url: https://cds.climate.copernicus.eu/api
key: SU-CLAVE-PERSONAL
```

4. Ejecutar

4.1. La forma general

```
./run.sh [PASO_DESDE] [PASO_HASTA] [MAX_MODELOS]
```

Los tres valores son opcionales. Sin ninguno, se ejecuta todo: pasos 00 00b 01 02 03 04 05 06 07, en ese orden. `PASO_DESDE` y `PASO_HASTA` limitan el rango de pasos. `MAX_MODELOS` procesa solo los primeros N modelos, lo que sirve para una prueba rápida.

Comando	Para qué sirve
<code>./run.sh 00 04 2</code>	Prueba rápida: todo hasta el regrillado, con solo 2 modelos. Úsela la primera vez para comprobar la instalación.
<code>./run.sh</code>	Corrida completa. La primera vez tarda del orden de 16 horas (Sección 5).
<code>./run.sh 06 07</code>	Repetir solo el control de calidad y el inventario (minutos).
<code>./run.sh 07 07</code>	Regenerar inventario, figuras, reporte y paquete sin tocar los datos.

Si la corrida se corta (corte de luz, de red, o se cierra la terminal), basta con volver a ejecutar el mismo comando. Cada paso detecta lo que ya está hecho y continúa desde ahí, sin repetir descargas.

4.2. Elegir el comando

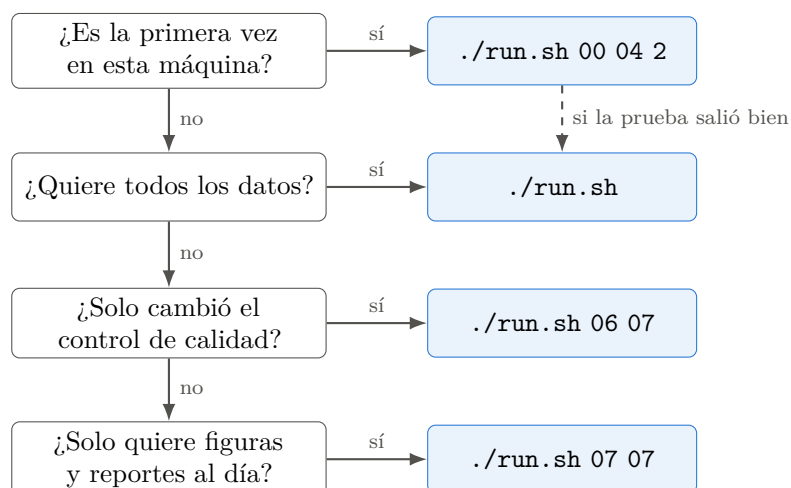


Figura 2: Qué comando usar según la situación.

4.3. Dejarlo corriendo en segundo plano

Como la corrida completa dura muchas horas, conviene lanzarla de modo que siga aunque se cierre la terminal:

```
nohup ./run.sh > logs/corrida.txt 2>&1 &
```

Para ver el avance en cualquier momento (se sale con **Ctrl+C**, sin detener la corrida):

```
tail -f logs/pipeline.log
```

4.4. Qué aparece en pantalla

Antes de empezar, `run.sh` muestra un **reporte previo** con lo que ya existe ([OK]) y lo que falta ([FALTA]), junto con una estimación de tiempo basada en corridas anteriores en la misma máquina. La Figura 3 muestra la salida real en la máquina de desarrollo, con los 40 modelos ya procesados.

```
$ ./run.sh
=== Estado antes de esta corrida (run.sh) ===
[OK] 00b config/models_seed_cmip6.csv (lista de modelos)
[OK] 01 data/interim/models_catalog_status.csv (catalogo ESGF)
[OK] 02 Descarga CMIP6: 160/160 combinaciones modelo x experimento
[OK] 04/05 Grilla comun + mascara: 160/160 archivos
[OK] 03 data/raw/ersstv5/ersstv5_region.nc (observado)
[OK] 06 data/processed/qc_report.csv (control de calidad)
[OK] 07 data/processed/models_inventory_final.csv (inventario final)
=====
# en una máquina nueva, cada línea diría por ejemplo:
[FALTA] 00b config/models_seed_cmip6.csv (lista de modelos) -- sin corridas previas en esta maquina, no
se puede estimar
```

Figura 3: Reporte previo de `run.sh` (salida real; la última línea ilustra el caso de una máquina nueva).

Durante la corrida pueden aparecer dos preguntas, ambas con límite de 15 segundos. No hace falta responderlas: si nadie contesta, el *pipeline* continúa solo.

Pregunta	Si no se responde
¿Buscar en nodos alternativos de ESGF y en Copernicus? [S/n]	Sí busca. Solo aparece si faltan modelos.
¿Regenerar las figuras que ya existen?	No regenera; conserva las figuras existentes.

Figuras desactualizadas. Si cambia el conjunto de modelos, las figuras que ya existen *no* se actualizan solas. Responda “s” a la segunda pregunta o borre la figura concreta en `figures/` antes de ejecutar `./run.sh 07 07`.

5. Cuánto tarda

La Figura 4 muestra lo que tardó cada paso en la máquina de desarrollo cuando tuvo que hacer todo el trabajo. La descarga domina: una primera corrida completa toma en total del orden de **16 horas**. Las corridas siguientes tardan segundos o minutos, porque cada paso omite lo que ya está hecho. El Paso 03 (ERSSTv5) no aparece porque no hay una medición de su primera descarga; es un único archivo de unos 160 MB.

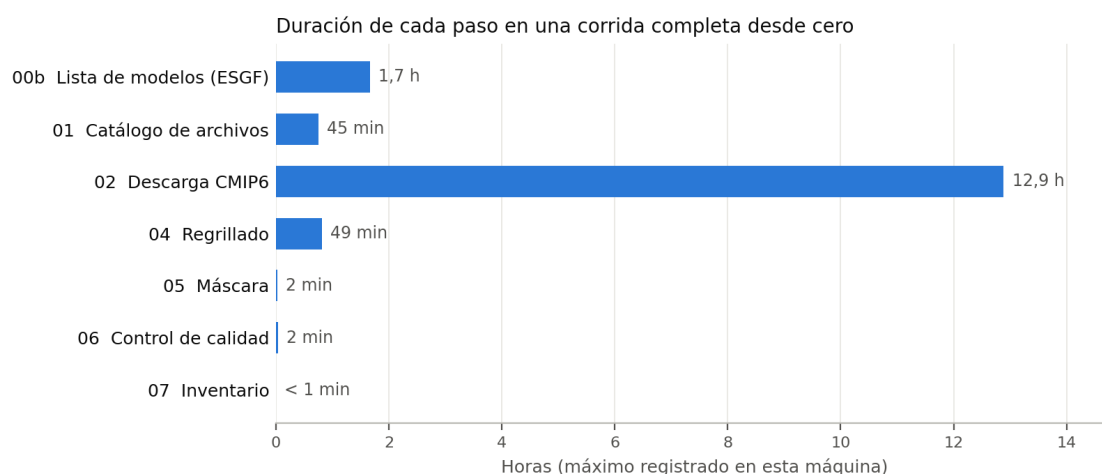


Figura 4: Duración máxima registrada por paso (`logs/step_timings.csv`). Los tiempos dependen de la conexión y del estado de los servidores de ESGF.

6. Dónde quedan los resultados

```
smn_toe/
|-- data/
| |-- raw/                                <- datos descargados, sin tocar (73 GB)
| |-- interim/                          <- catálogo de descargas e intermedios
| '-- processed/
|     |-- masked/                        <- ARCHIVOS FINALES: tos_<modelo>_<experimento>.nc
|     |                                     y ersstv5_region.nc (160 + 1 archivos)
|     |-- qc_report.csv                  <- control de calidad, una fila por archivo
|     '-- models_inventory_final.csv     <- lista final: selected=True / False
|-- figures/                             <- figuras generadas automáticamente
|-- logs/
| |-- pipeline.log                       <- todo lo que se mostró en pantalla
| '-- run_report_<fecha>.txt             <- resumen de estado de cada corrida
'-- informe/
    |-- model_registry.csv               <- código M001-M102, institución, país, resolución
    '-- entregable_update.tar.gz         <- paquete con todo lo necesario para el informe
```

6.1. Cómo revisar los resultados

¿Cuántos modelos quedaron seleccionados? (debe responder 40):

```
grep -c ",True$" data/processed/models_inventory_final.csv
```

¿Hubo algún archivo rechazado en el control de calidad? (debe responder 0):

```
grep -c ",FAIL," data/processed/qc_report.csv
```

Ver una tabla CSV ordenada en columnas:

```
column -s, -t < data/processed/models_inventory_final.csv | less
```

(se sale de `less` con la tecla `q`).

Ver qué contiene un archivo de datos:

```
conda activate smn_toe
cdo sinfo data/processed/masked/tos_ACCESS-CM2_historical.nc
```

El archivo `qc_report.csv` tiene una fila por archivo: modelo, experimento, TSM mínima y máxima (°C), número de meses, si pasa el rango físico (`ok_range`), si pasa la cobertura temporal (`ok_length`) y el estado final (PASS o FAIL). El reporte `logs/run_report_<fecha>.txt` clasifica los modelos de la lista de descarga en: descargados completos, con descarga parcial, procesados, descargados pero sin procesar y pendientes en fuentes alternativas.

7. Qué hace cada paso (detalle técnico)

Esta sección describe la lógica de cada script por su nombre y ruta, sin reproducir el código, que está disponible en el repositorio `smn_toe`. El flujograma de la Sección 6 del informe muestra el orden de los pasos; aquí se describe qué hace cada uno y qué controles incorpora.

7.1. Paso 00 – `scripts/00_setup_env.sh`

Verifica las dependencias (`cdo`, `wget`, `python3` con el paquete `requests`) y crea la estructura de carpetas de datos y registros. No instala nada: si falta una herramienta, falla al inicio con un mensaje claro. La adquisición de datos se hace por HTTP directo contra ESGF y NOAA PSL, porque el repositorio público de CMIP6 en la nube de Amazon está en formato Zarr, incompatible con CDO, y no existe un equivalente para ERSSTv5.

7.2. Paso 00b – `scripts/00b_build_model_list.py`

Consulta el índice del nodo principal de ESGF (ORNL) y enumera todos los modelos CMIP6 que publican `tos/0mon`, que son 102. Para cada uno exige `historical` (o su equivalente `hist-1950` de HighResMIP), que busca también en nodos alternativos si el nodo principal no lo tiene indexado. Exige además los tres escenarios SSP configurados y retiene solo las variantes de grilla (`grid_label`) que cubren los cuatro experimentos; entre ellas elige la de mayor `nominal_resolution`, es decir, la más gruesa, convertida a kilómetros si el vocabulario la expresa en grados. Los modelos que califican quedan en `config/models_seed_cmip6.csv`. Para todos los modelos inspeccionados, el script escribe además `informe/model_availability_priority.csv`, con la disponibilidad 0/1 por experimento. Antes de repetir el barrido completo, que tarda más de una hora, hace una consulta rápida del número de modelos disponibles y solo rehace el barrido si ese número cambió.

7.3. Paso 00c – `scripts/00c_check_paper_models.py` (diagnóstico manual)

Busca los nombres de modelos CMIP6 citados en los resúmenes de la bibliografía del proyecto (una carpeta local de archivos Markdown, externa al repositorio) y los compara con la lista del Paso 00b. Los citados que no están en la lista se escriben en `config/models_missing_from_esgf.csv`, como candidatos para las fuentes alternativas. No forma parte de la secuencia automática de `run.sh`.

7.4. Paso 01 – `scripts/01_query_esgf_catalog.py`

Para cada modelo de la lista, consulta el nodo LLNL de ESGF y determina el miembro de ensamble (`variant_label`) común a los cuatro experimentos, prefiriendo `r1i1p1f1`. Escribe el catálogo `data/interim/models_catalog_status.csv` (estado por modelo, fuente, miembro y número acumulado de intentos de búsqueda) y las direcciones de descarga en `data/interim/esgf_file_urls.json`. Si el catálogo ya existe, no vuelve a consultar ESGF; para forzar una consulta nueva hay que borrar ambos archivos.

La búsqueda de archivos recorre el índice de ESGF por páginas hasta traer la lista completa. Un corte de red a mitad del recorrido puede truncar la lista sin mostrar error: en pruebas, la misma búsqueda de EC-Earth3 `historical` devolvió 107, 121 o 165 archivos según la corrida (165 es el total real). Por eso, si el recorrido se interrumpe, se repite completo desde el inicio, aquí y en el Paso 02b.

7.5. Paso 02 – descarga en cascada

`scripts/02_download_all_sources.sh` encadena tres fuentes, cada una como respaldo automático de la anterior:

- (a) `02_download_cmip6_chunks.sh`: descarga HTTP directa con las direcciones del Paso 01. Cada dirección se reintenta tres veces, con una pausa breve, antes de pasar a la siguiente réplica. Es necesario para modelos que publican un archivo por año (hasta 165 archivos solo para `historical`), donde un corte transitorio en algún archivo es probable.

- (b) `02b_search_alt_esgf_nodes.py`: para los modelos que el nodo principal no resolvió, repite la búsqueda en ocho índices alternativos de la federación (CEDA, DKRZ, IPSL, NSC y NCI con la interfaz clásica, y ORNL, DKRZ y ESGF-West con la interfaz Metagrid), porque un modelo puede faltar en un índice por problemas de replicación y estar en otro.
- (c) `02c_download_copernicus_cds.py`: último recurso, solo para los modelos de `config/models_copernicus_available.csv`, la lista de modelos del conjunto `projections-cmip6` de Copernicus. Estar en la lista no garantiza que CDS tenga todos los experimentos; muchos pedidos de modelos poco replicados fallan con `RoocsValueError`. Requiere credenciales personales (`~/.cdsapirc`); sin ellas el paso se omite sin detener el resto.

Antes de las fuentes (b) y (c), si hay modelos pendientes y la sesión es interactiva, el script pregunta si se desea intentarlas. Espera 15 s y, sin respuesta, continúa como si fuera afirmativa, de modo que nunca bloquea una corrida desatendida. `run.sh` invoca este paso a través de `scripts/run_download_if_idle.sh`, que no lanza una segunda descarga si ya hay una en curso. Un modelo que agota (b) y (c) sin resolverse suma un intento en el catálogo; con tres intentos en corridas distintas deja de reintentarse hasta que se borre el catálogo.

7.6. Paso 03 – `scripts/03_download_ersstv5.sh`

Descarga ERSSTv5 desde NOAA PSL una sola vez y la recorta con CDO a la ventana de descarga: toda la franja tropical, 0°–360° y 30°S–30°N (`config/domains.yaml`).

7.7. Paso 04 – `scripts/04_process_to_common_grid.sh`

Para cada modelo y experimento:

- (1) Usa solo los archivos crudos de la variante de grilla registrada en el catálogo. Es necesario porque una misma carpeta puede contener archivos de dos grillas del mismo periodo (ocurre con CESM2, con `gn` y `gr`), y al fusionarlos cada mes quedaría duplicado.
- (2) Descarta toda variable distinta de `tos` antes de regrillar, porque algunos modelos de malla curvilínea, como IPSL-CM6A-LR, incluyen variables auxiliares que hacen fallar a CDO.
- (3) Calcula una vez por modelo los pesos de interpolación hacia la grilla de ERSSTv5: conservativa (`gencon`) para mallas no estructuradas, como la de AWI-CM-1-1-MR, y bilineal (`genbil`) para el resto. Si los pesos del modelo fallan en un experimento, se recalculan pesos propios para ese experimento.
- (4) Fusiona en el tiempo los fragmentos ya regrillados y recorta al rango de años del experimento.
- (5) Asigna calendario `standard` si falta el atributo.
- (6) Convierte Kelvin a grados Celsius según el valor y no según el atributo: resta 273,15 solo si la media en Niño 3.4 es mayor o igual a 100. La regla es necesaria porque hay archivos que declaran `degC` pero contienen valores en Kelvin, como uno de GISS-E2-1-G (`ssp585`).
- (7) Verifica que el resultado tenga exactamente la misma lista de meses (año y mes, no solo el conteo) que los archivos crudos de la grilla correcta; si no coincide, descarta el resultado para reprocesarlo en la corrida siguiente.

La comparación se hace contra lo que publican los archivos descargados y no contra un calendario ideal. Así, un modelo que publica permanentemente menos años de los pedidos se acepta tal cual: CAMS-CSM1-0 termina sus escenarios en 2099, e IITM-ESM empieza `historical` en 1900 y termina sus escenarios en 2098 o 2099.

7.8. Paso 05 – `scripts/05_apply_ocean_mask.py`

Construye una máscara océano-tierra derivada de ERSSTv5: un punto es océano si tiene al menos un valor válido en algún mes del registro. No usa la fracción de tierra propia de cada modelo (`sftlf`), de modo que la observación y todos los modelos comparten exactamente los mismos puntos válidos, condición necesaria para compararlos punto a punto. Se implementa en Python (`netCDF4/numpy`) porque la operación equivalente en CDO, aplicada sobre el valor de relleno, no propaga correctamente la máscara y marca como océano todo el dominio, incluida la tierra. Además aplica un filtro por valor físico: todo punto con $|\text{valor}| > 400$ pasa a NaN, independientemente de la máscara. El filtro es necesario porque FGOALS-f3-L contiene, en puntos que ERSSTv5 considera océano, un valor de relleno ($\sim 10^{35}$) que el regrillado no reconoce como faltante; gracias a este filtro el modelo puede incorporarse al inventario.

7.9. Paso 06 – `scripts/06_qc_checks.py`

Control de calidad automático de cada archivo procesado:

- **Rango físico:** la TSM debe estar entre -2 y 45 °C. El límite superior considera que la franja tropical incluye mares marginales cálidos: en el mar Rojo y el golfo Pérsico algunos modelos alcanzan, de forma plausible, entre 39 y 41 °C hacia 2100 en SSP3-7.0 y SSP5-8.5.
- **Cobertura temporal:** en los escenarios SSP se admite hasta un 10 % menos de los meses esperados; en `historical` se exige que el registro llegue más allá de 1950, porque algunos modelos válidos no empiezan en 1850 (IITM-ESM empieza en 1900).

El resultado se escribe en `data/processed/qc_report.csv`. Las métricas se recalculan para todos los archivos en cada corrida, sin caché: un resultado guardado por nombre de archivo podría quedar desactualizado cuando un archivo se reprocesa. Como el control tarda menos de un minuto, recalcular siempre es la opción más segura. La exactitud de la cobertura la garantiza la verificación mes a mes del Paso 04; en la corrida final, los 40 `historical` terminan en 2014 y todos los archivos tienen sus meses completos.

7.10. Paso 07 – `scripts/07_build_inventory_report.py`

Combina el resultado del Paso 06 con la resolución de grilla (obtenida con `cdo griddes`) en `data/processed/models_inventory_final.csv`. Un modelo queda `selected=True` solo si aprueban el control de calidad los cuatro experimentos requeridos. Los cuatro se exigen explícitamente, en lugar de compararlos con los experimentos intentados, para que un modelo con descarga parcial no pueda quedar seleccionado aunque lo descargado apruebe.

7.11. Reporte de estado – `scripts/generate_status_report.py`

Al final de cada corrida de `run.sh`, sin importar el rango de pasos, se escribe `logs/run_report_<fecha>.txt`, que clasifica cada modelo como descargado completo, con descarga parcial, procesado, descargado pero sin procesar, o pendiente de fuentes alternativas.

8. Problemas frecuentes

Mensaje o síntoma	Qué hacer
<code>conda: command not found</code>	Miniconda no está instalado, o la terminal se abrió antes de instalarlo. Cierre y abra la terminal; si persiste, repita el Paso 1 de la instalación.
FALTA <code>cdo</code> u otra herramienta al inicio	El entorno no se creó. Ejecute <code>conda env create -f environment.yml</code> .
Permission denied al ejecutar <code>./run.sh</code>	Ejecute <code>chmod +x run.sh</code> una vez.
“Descarga ya en curso en otro proceso”	Ya hay una descarga corriendo, por ejemplo en segundo plano. Es normal: espere a que termine y vuelva a ejecutar.
“se omite” en Copernicus / falta <code>.cdsapirc</code>	No hay credenciales de Copernicus. Es opcional; el resto del <i>pipeline</i> sigue.
La descarga se detuvo o se cortó la red	Vuelva a ejecutar el mismo comando; continúa donde quedó.
Un modelo “agotó 3 intentos”	No se volverá a buscar automáticamente. Para reintentar desde cero, borre <code>data/interim/models_catalog_status.csv</code> y <code>data/interim/esgf_file_urls.json</code> y ejecute de nuevo.
Disco lleno (No space left on device)	Libere espacio; con <code>du -sh data/*</code> se ve qué ocupa más. No borre <code>data/processed/</code> .
Las figuras no reflejan los últimos cambios	Ver el aviso de la Sección 4.

9. Qué se espera hacer con estos datos

El Entregable 1 deja listo el dato. Los tres entregables siguientes lo usan en este orden (Figura 5): el Entregable 2 evalúa qué modelos representan mejor el ENOS, el Entregable 3 proyecta los cambios futuros y el Entregable 4 estima la incertidumbre y el tiempo de emergencia.

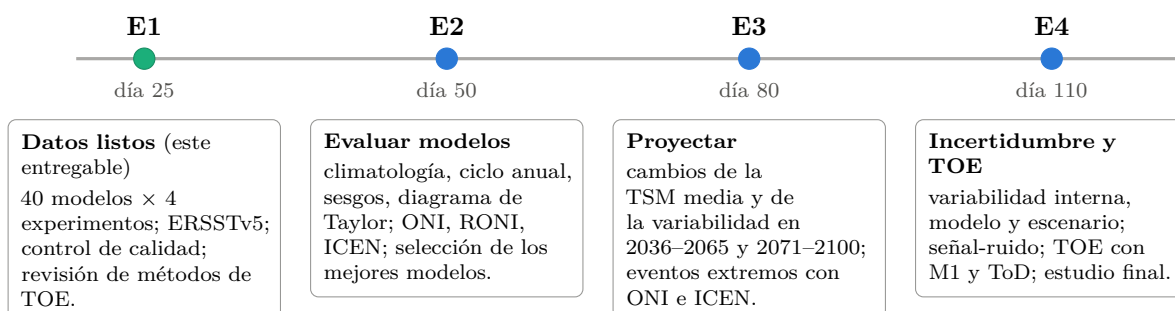


Figura 5: Hoja de ruta del servicio. Los días se cuentan desde la notificación de la orden de servicio, según los términos de referencia.

Para el Entregable 2 se usan directamente los archivos de `data/processed/masked/` y la lista de `models_inventory_final.csv`: no hace falta volver a descargar nada.

10. Estructura del repositorio

Para quien necesite ubicar un script concreto:

```
smn_toe/
|-- run.sh                      # punto de entrada único
|-- environment.yml            # entorno conda (python, cdo, netcdf4, ...)
|-- README.md                  # documentación técnica detallada
|-- config/
|   |-- domains.yaml           # cajas Niño 3.4 / Niño 1+2 y ventana de descarga
|   |-- periods.yaml           # periodos y escenarios
|   |-- models_missing_from_esgf.csv # candidatos de fuente alternativa (00c)
|   |-- models_no_encontrado_44.csv # modelos sin algún experimento en ninguna fuente
|   |-- models_copernicus_available.csv # modelos ofrecidos por Copernicus (02c)
|   |-- models_copernicus_ssp245_whitelist.csv # referencia histórica, sin uso
|   |-- CMIP6_institution_id.json # vocabulario oficial CMIP6: institución -> país
|-- scripts/
|   |-- 00_setup_env.sh         # verifica herramientas y crea carpetas
|   |-- 00b_build_model_list.py # universo de modelos (ESGF, nodo ORNL)
|   |-- 00c_check_paper_models.py # diagnóstico manual, no lo ejecuta run.sh
|   |-- 01_query_esgf_catalog.py # catálogo de archivos (ESGF, nodo LLNL)
|   |-- 02_download_all_sources.sh # descarga en cascada 02a -> 02b -> 02c
|   |-- 02_download_cmip6_chunks.sh, 02b_search_alt_esgf_nodes.py,
|   |   02c_download_copernicus_cds.py, run_download_if_idle.sh
|   |-- 03_download_ersstv5.sh
|   |-- 04_process_to_common_grid.sh
|   |-- 05_apply_ocean_mask.py
|   |-- 06_qc_checks.py
|   |-- 07_build_inventory_report.py
|   |-- build_model_registry.py # códigos M001-M102
|   |-- generate_status_report.py # logs/run_report_<fecha>.txt
|   |-- preflight_report.py      # reporte previo con estimación de tiempo
|   |-- pipeline_config.py, pipeline_timing.py # configuración y tiempos
|   |-- bundle_deliverable_update.py # paquete .tar.gz
|   |-- plot_*.py                # las cinco figuras automáticas
-- data/, figures/, logs/, informe/ # salidas (Sección 6)
```